Softwareparadigms SS 2025, Assignment 1

Assignments are solved in teams of three persons. Groups remain the same for the whole term. You can find the group registration in the TeachCenter.

Submission: The solution has to be submitted via the SAI student git. Using the provided latex template is mandatory (No handwritten solutions!). Please check in the LaTeX source **as well as** the compiled PDF. Both files have to follow the naming convention '`sol<assignment number>_<group number>.<ext>`' (e.g. '`sol1_23.tex`').
Deadline for Group-/GitLab-Registration: 19.03.2025, 23:59 Uhr
Deadline Assignment 1: 02.04.2025, 17:00 Uhr

Task 1: GitLab - Registration (0 P.)

Each group member is required to sign up to the SAI student git via the TUG SSO Login using the following link **before 19.03.2025**: `https://git-students.sai.tugraz.at`. The group repositories will be created after the group registration deadline. This is a prerequisite for handing in solutions.

Task 2: Grammars (1.5 P.)

Define a the following languages as 4-tuple grammars. Make sure that the language is as restrictive as possible.

a) $L = \{\underline{\mathtt{a}}(\underline{\mathtt{c}}|\underline{\mathtt{b}})^*\underline{\mathtt{c}}^n \mid n > 0\}$

b) $L = \{\underline{\mathtt{x}}^m(\underline{\mathtt{a}}\,\underline{\mathtt{b}}|\underline{\mathtt{b}}\,\underline{\mathtt{a}})^n\underline{\mathtt{x}} \mid n > 0,\ m > 1\}$

c) $L = \{\underline{\mathtt{a}}^{2n+2}\underline{\mathtt{b}}\,\underline{\mathtt{c}}^*(\underline{\mathtt{b}}\,\underline{\mathtt{b}}|\underline{\mathtt{d}})^n \mid n > 0\}$

a)

$$L = \{\underline{\mathtt{a}}(\underline{\mathtt{c}}|\underline{\mathtt{b}})^*\underline{\mathtt{c}}^n \mid n > 0\}$$

$$\begin{aligned} V_N &= \{S, A, B\} \\ V_T &= \{\underline{\mathtt{a}}, \underline{\mathtt{b}}, \underline{\mathtt{c}}\} \\ S &= S \end{aligned}$$

Φ :

$$\begin{aligned} S &\to \mathtt{a}A \\ A &\to \mathtt{b}A \\ A &\to \mathtt{c}A \\ A &\to B \\ B &\to \mathtt{c}B \\ B &\to \mathtt{c} \end{aligned}$$

b)

$$L = \{\underline{\mathtt{x}}^m(\underline{\mathtt{a}}\,\underline{\mathtt{b}}|\underline{\mathtt{b}}\,\underline{\mathtt{a}})^n\underline{\mathtt{x}} \mid n > 0,\ m > 1\}$$

$$\begin{aligned} V_N &= \{S, A, B, C\} \\ V_T &= \{\underline{\mathtt{a}}, \underline{\mathtt{b}}, \underline{\mathtt{x}}\} \\ S &= S \end{aligned}$$

Φ :

$$\begin{aligned} S &\to \mathtt{x}\,\mathtt{x}\,A \\ A &\to \mathtt{x}\,A \\ A &\to B \\ B &\to \mathtt{a}\,\mathtt{b}\,B \\ B &\to \mathtt{b}\,\mathtt{a}\,B \\ B &\to \mathtt{a}\,\mathtt{b}\,C \\ B &\to \mathtt{b}\,\mathtt{a}\,C \\ C &\to \mathtt{x} \end{aligned}$$

grammar not regular

c)

$$L = \{\underline{\mathtt{a}}^{2n+2}\underline{\mathtt{b}}\,\underline{\mathtt{c}}^*(\underline{\mathtt{b}}\,\underline{\mathtt{b}}|\underline{\mathtt{d}})^n \mid n > 0\}$$

$$\begin{aligned} V_N &= \{S, A, B, C, D\} \\ V_T &= \{\underline{\mathtt{a}}, \underline{\mathtt{b}}, \underline{\mathtt{c}}, \underline{\mathtt{d}}\} \\ S &= S \end{aligned}$$

$$\begin{array}{lll} & \Phi: & \\ S & \to \mathtt{a}\,\mathtt{a}\,A & \text{effectively unbound occurrences of n} \\ A & \to \mathtt{a}\,\mathtt{a}\,A & \\ A & \to B & \\ B & \to \mathtt{b}\,C & \\ C & \to \mathtt{c}\,C & \\ C & \to D & \\ D & \to \mathtt{b}\,\mathtt{b}\,D & \\ D & \to \mathtt{d}\,D & \\ D & \to \mathtt{b}\,\mathtt{b} & \\ D & \to \mathtt{d} & \end{array}$$

Task 3: Chomsky hierarchy (3 P.)

For each grammar, mark all grammar types it belongs to with a cross "`X`". Explain why you made this decision and state the relevant production rule.

Grammars	G	G_{sens}	G_{free}	G_{reg}	rule	reason
$V_T = \{\underline{\texttt{val}}, \underline{\texttt{float}}, \underline{\texttt{int}}, \underline{\texttt{(}}, \underline{\texttt{)}}, \underline{\texttt{x}}, \underline{\texttt{y}}\}$ $V_N = \{A, C, D, S\}$ $S = S$ Φ: $S \to \underline{\texttt{val}}A$ $A \to \underline{\texttt{x}}D$ $D \to \underline{\texttt{int}}\underline{\texttt{(}}C\underline{\texttt{)}}$ $\underline{\texttt{int}}\underline{\texttt{(}}C\underline{\texttt{)}} \to \underline{\texttt{float}}\underline{\texttt{(}}C\underline{\texttt{)}}$ $C \to \underline{\texttt{y}}$	X	X	-	-	$\underline{\texttt{int}}\underline{\texttt{(}}C\underline{\texttt{)}} \to \underline{\texttt{float}}\underline{\texttt{(}}C\underline{\texttt{)}}$	Not a context-free grammar, because terminals are not allowed on the left-hand side. However, the condition $\lvert\alpha\rvert \leq \lvert\beta\rvert$ is satisfied, so the grammar is context-sensitive.
$V_T = \{\underline{\texttt{a}}, \underline{\texttt{b}}, \underline{\texttt{c}}, \underline{\texttt{d}}\}$ $V_N = \{A, B\}$ $S = A$ Φ: $A \to \underline{\texttt{a}}A\underline{\texttt{b}}$ $A \to B\underline{\texttt{cd}}$ $B \to \underline{\texttt{c}}B$ $B \to \epsilon$	X	X	x	-	$A \to \underline{\texttt{a}}A\underline{\texttt{b}}$ $A \to B\underline{\texttt{cd}}$	Not a regular grammar, since this requires that the right-hand side is of the form aA, a, or ε, where $a \in V_T$, $A \in V_N$
$V_T = \{\underline{\texttt{;}}, \underline{\texttt{=}}, \underline{\texttt{let}}, \underline{\texttt{mut}}, \underline{\texttt{x}}, \underline{\texttt{y}}, \underline{\texttt{z}}\}$ $V_N = \{A, B, S, X\}$ $S = S$ Φ: $S \to \underline{\texttt{let}}\ \underline{\texttt{mut}}A\underline{\texttt{=}}B\underline{\texttt{;}}$ $A \to \underline{\texttt{x}}$ $\underline{\texttt{mut}}A \to \underline{\texttt{x}}$ $\underline{\texttt{x}} \to \underline{\texttt{z}}$ $B \to \underline{\texttt{y}}$	-	-	-	-	$\underline{\texttt{x}} \to \underline{\texttt{z}}$	Invalid, because terminal symbols may not stand alone on the on the left-hand side. α must have at least one non-terminal symbol must be included.
$V_T = \{\underline{\texttt{mut}}, \underline{\texttt{4}}, \underline{\texttt{2}}, \underline{\texttt{x}}, \underline{\texttt{y}}, \underline{\texttt{=}}\}$ $V_N = \{C, E, T, V\}$ $S = T$ Φ: $T \to \underline{\texttt{mut}}V$ $V \to \underline{\texttt{x}}C \mid \underline{\texttt{y}}C$ $C \to \underline{\texttt{=}}E$ $E \to \underline{\texttt{4}}E$ $E \to \underline{\texttt{2}}$	X	X	X	X	-	All conditions for a regular grammar are satisfied, since the production rules are of the form $X \to aA$, $X \to a$, or $X \to \varepsilon$, where $X \in V_N$, $a \in V_T$, and $A \in V_N$.
$V_T = \{\underline{\texttt{a}}, \underline{\texttt{c}}, \underline{\texttt{d}}, \underline{\texttt{x}}\}$ $V_N = \{P, X, Y\}$ $S = X$ Φ: $X \to \underline{\texttt{a}}Y$ $Y \to \underline{\texttt{c}}Y \mid \underline{\texttt{d}}$ $\underline{\texttt{c}}Y \to P$ $P \to \underline{\texttt{x}}P$ $P \to \epsilon$	X	-	-	-	$\underline{\texttt{c}}Y \to P$	Unrestricted, since context-sensitive grammar requires $\lvert\alpha\rvert \leq \lvert\beta\rvert$, where the cardinality describes the number of terminal and nonterminal symbols.
$V_T = \{\underline{\texttt{;}}, \underline{\texttt{.}}, \underline{\texttt{-}}, \underline{\texttt{+}}\}$ $V_N = \{E, M, N, S\}$ $S = S$ Φ: $S \to \underline{\texttt{+}}E$ $E \to \underline{\texttt{+}}E \mid \underline{\texttt{.}}M$ $M \to \underline{\texttt{-}}N$ $N \to E\underline{\texttt{;}}$ $E \to \epsilon$	X	X	X	-	$N \to E\underline{\texttt{;}}$	Not a regular grammar, since this requires that the right-hand side is of the form aA, a, or ε, where $a \in V_T$, $A \in V_N$

Task 4: Defining Grammars (2 P.)

Write a grammar for defining simple Bash pipes. This is a simplified representation of a sequence of commands that are executed sequentially, with the output of one command passed as input to the next using the pipe operator `|`. Additionally, the grammar should support CmdLine execution using the logical AND and OR operators, `&&` and `||` respectively. Here is an example:

```
ls -l | grep "txt" | wc -l && echo "Success :)" || echo "Failed :("
```

The following points should be considered for the grammar definition:

- A word of the language starts with a command.
- Commands can be connected using the pipe operator `|`. The output of the command on the left of the pipe is passed as input to the command on the right. There can be any number of commands connected by pipes.
- After a command or a sequence of piped commands, CmdLine execution can be specified using the logical AND (`&&`) or OR (`||`) operators.
  - The operator `&&` executes the command on its right only if the command on its left succeeds.
  - The operator `||` executes the command on its right only if the command on its left fails.
  - CmdLine execution can be chained. For example, `command1 && command2 || command3` means that `command2` will be executed if `command1` succeeds, and `command3` will be executed if either `command1` fails or `command2` fails.
  - There must not be another pipe (`|`) after any CmdLine.
- The grammar should support the following commands: `ls`, `grep`, `wc`, `echo`, and `cat`. These commands can be used in any combination within the pipeline. Depending on the command, commands may or must be followed by arguments (e.g. `wc -l`). The following restrictions on arguments hold:
  - The `ls` command can have any number of arguments, each being either `-l` or a directory path.
  - The `grep` command must have exactly one argument which is `"txt"`.
  - The `wc` command may have `-l` as an argument.
  - The `echo` command may have `"Success :)"` or `"Failed :("` as an argument, never both.
  - The `cat` command requires a positive number of file paths as arguments.
- Paths:
  - A directory path is an alternating sequence of directories and `/` terminals, ending with `/`.
  - A directory consists of at least one character. Characters are represented by the terminal `char`.
  - A file path is either a directory path followed by a file or just a file.
  - A file consists of at least one character and optionally a `.` terminal.
- The grammar should allow for any combination of these commands, connected by pipes and CmdLine operators, to form a valid pipeline.

Complete the following tasks:

a) Define the grammar G_{Bash} formally as a context-free grammar. Avoid left recursion and common prefixes on identical left-hand sides.

b) Can this grammar also be represented as a regular grammar? Justify your answer.

4.0.1 Solution

a) Define context-free grammar

Definition: $G_{\text{Bash}} = (V_N, V_T, S, \Phi)$ where:

$$V_T = \{\texttt{ls}, \texttt{grep}, \texttt{wc}, \texttt{echo}, \texttt{cat}, \texttt{txt}, \texttt{-l}, \texttt{'Success :)'}, \texttt{'Failed :('}, \texttt{|}, \texttt{\&\&}, \texttt{||}, \texttt{/}, \texttt{char}, \texttt{_}\}.$$

$$\begin{aligned} V_N = \{ & \texttt{S}, \texttt{Pipeline}, \texttt{PipContent}, \texttt{CmdLine}, \texttt{CmdLineContentCommands}, \texttt{LS}, \texttt{GREP}, \texttt{WC}, \texttt{ECHO}, \\ & \texttt{CAT}, \texttt{CondOperator}, \texttt{LSArg}, \texttt{LSArgs}, \texttt{WCArgs}, \texttt{ECHOArgs}, \texttt{CATArgs}, \texttt{DirPath}, \\ & \texttt{Directory}, \texttt{DirContent}, \texttt{DirSlash}, \texttt{FilePath}, \texttt{File}, \texttt{FileContent}, \texttt{OptionalChar}\} \end{aligned} \tag{1}$$

$$S = \text{CmdLine}$$

$$\Phi :$$

CmdLine	$\rightarrow$	Pipeline CmdLineContent
CmdLineContent	$\rightarrow$	CondOperator Pipeline CmdLineContent | ϵ
CondOperator	$\rightarrow$	`&&` | `||`
Pipeline	$\rightarrow$	Commands PipContent
PipContent	$\rightarrow$	ϵ | '|' Commands PipContent
Commands	$\rightarrow$	LS | GREP | WC | ECHO | CAT
LS	$\rightarrow$	`ls` LSArgs
LSArgs	$\rightarrow$	ϵ | LSArg LSArgs
LSArg	$\rightarrow$	`-l` | DirPath
GREP	$\rightarrow$	`grep txt`
WC	$\rightarrow$	`wc` WCArgs
WCArgs	$\rightarrow$	ϵ | `-l`
ECHO	$\rightarrow$	`echo` ECHOArgs
ECHOArgs	$\rightarrow$	ϵ | `''Success :)''` | `''Failed :(''`
CAT	$\rightarrow$	`cat` CATArgs
CATArgs	$\rightarrow$	FilePath | FilePath CATArgs
FilePath	$\rightarrow$	DirPath File | File
DirPath	$\rightarrow$	DirSlash Directory | DirSlash Directory DirPath
DirSlash	$\rightarrow$	`/`
Directory	$\rightarrow$	`char` DirContent
DirContent	$\rightarrow$	ϵ | `char` DirContent
File	$\rightarrow$	`char` FileContent
FileContent	$\rightarrow$	ϵ | `char` FileContent | `OptionalChar` FileContent
OptionalChar	$\rightarrow$	`_` ?

No pipe after any conditional!

same prefix

"/" is also a valid DirPath

what is this?
if this is a ".", then multiple dots in one file are not valid
if this is not a ".", then one dot is possible in files

b) Can grammar be represented as a regular grammar?
Yes, it can be represented by a regular grammar. A regular grammar has productions of the form:

$$X \rightarrow aY,\ X \rightarrow a \ or\ X \rightarrow \epsilon,$$

where we can see that a terminal is followed by non-terminal and optionally terminal. Our Bash-style language—consisting of commands, flags, and operators, can be handled by a finite automaton. Because a finite automaton suffices, it follows that the language is regular.

Regular Grammar representation:

$$V_T = \{\texttt{ls}, \texttt{grep}, \texttt{wc}, \texttt{echo}, \texttt{cat}, \texttt{txt}, \texttt{-l}, \texttt{'Success :)'}, \texttt{'Failed :('}, \texttt{|}, \texttt{\&\&}, \texttt{||}, \texttt{/}, \texttt{char}, \texttt{_}\}.$$

$$V_N = \{S, R, P, P', C, \text{LsCmd}, \text{LsState}, \text{DirSt}, \text{EndDir}, \text{GrepCmd}, G_1, \text{WcCmd}, W, \text{EchoCmd}, E, \text{CatCmd}, \text{Cat}$$

$$\begin{aligned}
S &\rightarrow P\ R \\
R &\rightarrow \texttt{\&\&}\ P\ R \mid \texttt{||}\ P\ R \mid \epsilon \\
P &\rightarrow C\ P' \\
P' &\rightarrow \texttt{'|'}\ C\ P' \mid \epsilon \\
C &\rightarrow \text{LsCmd} \mid \text{GrepCmd} \mid \text{WcCmd} \mid \text{EchoCmd} \mid \text{CatCmd} \\
\text{LsCmd} &\rightarrow \texttt{ls}\ \text{LsState} \\
\text{LsState} &\rightarrow \texttt{-l}\ \text{LsState} \mid \texttt{/}\ \text{DirSt} \mid \epsilon \\
\text{DirSt} &\rightarrow \texttt{char}\ \text{DirSt} \mid \texttt{_}\ \text{DirSt} \mid \texttt{/}\ \text{DirSt} \mid \text{EndDir} \\
\text{EndDir} &\rightarrow \text{LsState} \\
\text{GrepCmd} &\rightarrow \texttt{grep}\ G_1 \\
G_1 &\rightarrow \texttt{txt} \\
\text{WcCmd} &\rightarrow \texttt{wc}\ W \\
W &\rightarrow \texttt{-l} \mid \epsilon \\
\text{EchoCmd} &\rightarrow \texttt{echo}\ E \\
E &\rightarrow \texttt{'Success :)'} \mid \texttt{'Failed :('} \mid \epsilon \\
\text{CatCmd} &\rightarrow \texttt{cat}\ \text{CatSt} \\
\text{CatSt} &\rightarrow \text{FileSt}\ \text{CatTail} \\
\text{CatTail} &\rightarrow \text{FileSt}\ \text{CatTail} \mid \epsilon \\
\text{FileSt} &\rightarrow \texttt{/}\ \text{DirSt2} \mid \text{BaseSt} \\
\text{DirSt2} &\rightarrow \texttt{char}\ \text{DirSt2} \mid \texttt{_}\ \text{DirSt2} \mid \texttt{/}\ \text{DirSt2} \mid \text{EndDir2} \\
\text{EndDir2} &\rightarrow \text{BaseSt} \\
\text{BaseSt} &\rightarrow \texttt{char}\ \text{BaseSt} \mid \texttt{_}\ \text{BaseSt} \mid \epsilon
\end{aligned}$$

As we can see the main difference between context free grammar and regular grammar is that each production rule in regular grammar has exactly one terminal, followed by at the most one non-terminal or no terminal at all. Meanwhile, in a context-free grammar, a single production can include multiple terminals and multiple non-terminals in any order, allowing more complex patterns such as nesting.

Task 5: Rewriting Grammars & Parse-Trees (2 P.)

Given is the grammar G_{zig}, which describes the pattern matching in Zig in a simplified way.

$$\begin{aligned}
V_N &= \{S, A, B, C, D, E, T, Z\} \\
V_T &= \{\ \underline{\texttt{switch}},\ \underline{\texttt{x}},\ \underline{\texttt{y}},\ \underline{\texttt{0}},\ \underline{\texttt{1}},\ \underline{\texttt{2}},\ \underline{\texttt{3}}, \underline{\texttt{(}},\ \underline{\texttt{)}},\ \underline{\texttt{\{}},\ \underline{\texttt{\}}}, \\
&\quad \underline{\texttt{=>}},\ \underline{\texttt{,}},\ \underline{\texttt{...}},\ \underline{\texttt{calculate}}, \underline{\texttt{;}},\ \underline{\texttt{else}}, \\
&\quad \underline{\texttt{@compileError("Invalid")}},\ \}
\end{aligned}$$

$$S = S$$

$$\Phi :$$

$$
\begin{array}{lcl}
S & \rightarrow & \underline{\texttt{switch}}\ A \\
A & \rightarrow & \underline{\texttt{(x)}}\ B \\
A & \rightarrow & \underline{\texttt{(y)}}\ B \\
B & \rightarrow & \underline{\texttt{\{}}\ C\ \underline{\texttt{\};}} \\
C & \rightarrow & D\ \underline{\texttt{=>}}\ \underline{\texttt{calculate(}}T\underline{\texttt{),}}\ C \\
C & \rightarrow & D\ \underline{\texttt{=>}}\ T\underline{\texttt{,}}\ C \\
C & \rightarrow & D\ \underline{\texttt{=>}}\ Z\underline{\texttt{,}}\ C \\
C & \rightarrow & D\ \underline{\texttt{=>}}\ \underline{\texttt{@compileError("Invalid"),}}C \\
C & \rightarrow & \epsilon \\
D & \rightarrow & \underline{\texttt{else}} \\
D & \rightarrow & Z \\
D & \rightarrow & Z\underline{\texttt{...}}Z \\
D & \rightarrow & Z\underline{\texttt{,}}E \\
E & \rightarrow & E\underline{\texttt{,}}Z \\
E & \rightarrow & Z \\
T & \rightarrow & \underline{\texttt{x}} \mid \underline{\texttt{y}} \\
Z & \rightarrow & \underline{\texttt{0}} \mid \underline{\texttt{1}} \mid \underline{\texttt{2}} \mid \underline{\texttt{3}}
\end{array}
$$

a) Why is the given grammar not suitable for efficient top-down parsing? Which rules are the reason for that?

b) Rewrite the grammar G_{zig} in a way, that it's suitable for efficient Top-Down Parsing. The new language $L(G_{zig_1})$ should be equivalent to the original language $L(G_{zig})$. Make sure that the number of production rules is as large as necessary and as small as possible.

c) Create a Parse-Tree for the following word of the language $L(G_{zig_1})$:

```
switch(x){
    1,2 => calculate(x),
    else => @compileError("Invalid"),
  };
```

Use your transformed grammar G_{zig_1}.

5.1 Solution:

5.1.1 a)

Top down parsing often runs into 2 problems, which are both present in the given grammar:

1. Left Recursion causing infinite loop:

1 of the productions for E is:

$$E \rightarrow E, Z$$

This is left recursion because the right hand side (production) starts with the same symbol as the left hand side. In top down parsing, the parser would call the same procedure repeatedly without consuming any input, esentially causing infinite loop.

2. Identical prefixes:

Some of the productions of A, C and of D have identical prefixes, namely:

$$A \rightarrow \quad (x)B \quad | \quad (y)B$$ good!

$$C \rightarrow \quad D \Rightarrow \text{calculate}(T), C \quad | \quad D \Rightarrow T, C \quad | \quad D \Rightarrow Z, C \quad | \quad D \Rightarrow \text{@compileError(Ïnvalid“)}, C$$

$$D \rightarrow \quad Z \quad | \quad Z...Z \quad | \quad Z, E$$

For these A productions, they all have the same prefix '(' and postfix ')B'. For C productions, prefix 'D ⇒ ' and postfix ',C'. For D production is the prefix 'Z'.
If the lookahead symbol is 1 digit, the parser cannot decide which production to apply without additional lookahead, since it has to look past the prefix to know which production to choose.

5.1.2 b)

First lets remove the left recursion by introducing a new nonterminal E' to handle the recursive part

$$E \rightarrow ZE'$$
$$E' \rightarrow, ZE' \mid \epsilon$$

Now lets handle problem with same prefix in production of C, this is done by grouping all parts that are not the prefix into a new nonterminal C'. Also, since most of them also ends with the same letters ',C', we can exclude this from C' :

$$C \rightarrow \quad D \Rightarrow C', C \quad | \quad \epsilon$$

$$C' \rightarrow \quad \text{calculate}(T) \quad | \quad T \quad | \quad Z \quad | \quad \text{@compileError(Ïnvalid“)}$$

Same principle for A:

$$A \rightarrow (A')B$$
$$A' \rightarrow x \mid y$$

But this already exists as T, so we can just use that.

Finally lets handle prefix for D, same principle as above:

$$D \rightarrow \quad \text{else} \quad | \quad ZD'$$

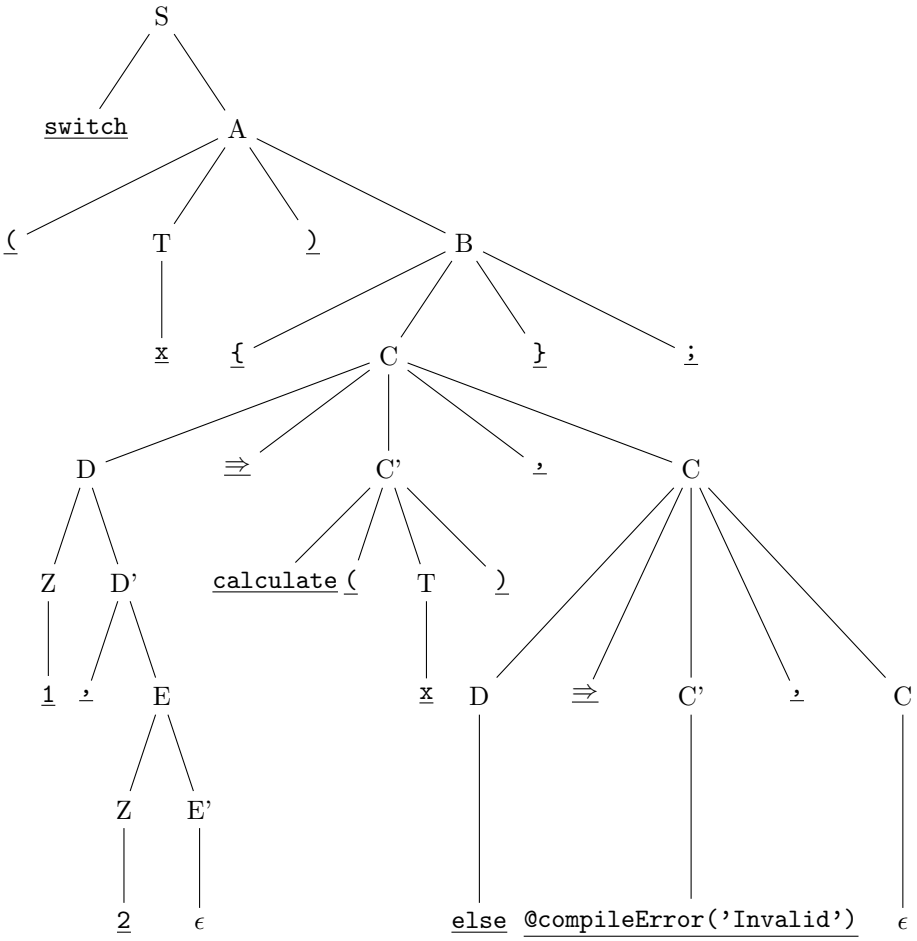
$$D' \rightarrow \quad ...Z \quad | \quad ,E \quad | \quad \epsilon$$

The complete transformed grammar G_{zig1} is:

$$V_N = \{S, A, B, C, C', D, D', E, E', T, Z\}$$
$$V_T = \{\text{switch}, x, y, 0, 1, 2, 3, (,), \{, \}, \Rightarrow, , , ..., \text{calculate}, ; , \text{else}, \text{@compileError(Ïnvalid“)}\}$$
$$S = S$$
$$\Phi :$$
$$S \rightarrow \text{switch } A$$
$$A \rightarrow (T)B$$
$$B \rightarrow \{C\};$$
$$C \rightarrow D \Rightarrow C', C \mid \epsilon$$
$$C' \rightarrow \text{calculate}(T) \mid T \mid Z \mid \text{@compileError(Ïnvalid“)}$$
$$D \rightarrow \text{else} \mid ZD'$$
$$D' \rightarrow ...Z \mid, E \mid \epsilon$$
$$E \rightarrow ZE'$$
$$E' \rightarrow, ZE' \mid \epsilon$$
$$T \rightarrow x \mid y$$
$$Z \rightarrow 0 \mid 1 \mid 2 \mid 3$$

Now thats efficient!

5.1.3 c)


S
switch
A
(
T
)
B
x
{
C
}
;
D
⇒
C'
,
C
Z
D'
calculate
(
T
)
1
,
E
x
D
⇒
C'
,
C
Z
E'
2
ϵ
else
@compileError('Invalid')
ϵ

Task 6: Recursive-Descent Parser (1.5 P.)

Given the following grammar: $G_6 =$

$$V_N = \{S, A, B, F, U, X, Y, Z\}$$

$$V_T = \{\underline{\mathtt{a}},\ \underline{\mathtt{d}},\ \underline{\mathtt{e}},\ \underline{\mathtt{f}},\ \underline{\mathtt{n}},\ \underline{\mathtt{x}},\ \underline{\mathtt{z}}\}$$

$$S = S$$

$$\Phi :$$

$$\begin{array}{lcl}
S & \rightarrow & A\underline{\mathtt{a}}B \\
A & \rightarrow & \underline{\mathtt{a}} \\
A & \rightarrow & \underline{\mathtt{a}}A \\
A & \rightarrow & \underline{\mathtt{a}}X \\
X & \rightarrow & X\underline{\mathtt{e}} \\
X & \rightarrow & \epsilon \\
B & \rightarrow & \underline{\mathtt{z}}UZ \\
U & \rightarrow & \underline{\mathtt{z}} \\
U & \rightarrow & \underline{\mathtt{za}} \\
U & \rightarrow & \epsilon \\
Z & \rightarrow & FXY\underline{\mathtt{end}} \\
F & \rightarrow & \underline{\mathtt{f}} \\
F & \rightarrow & \epsilon \\
Y & \rightarrow & \underline{\mathtt{x}}Y \\
Y & \rightarrow & \epsilon
\end{array}$$

- Define a recursive-decent parser for the language $L(G_6)$.
- You are may apply grammar transformations to make the parsing easier.
- Use one of the following programming languages: Python3, Scala, Clojure, Rust, C/C++, Zig or Lean3.
- The programm is called via `./task6.<ext> <word>`
- If the word **is** in the language, the output of the program must be: `"True"`.
- If the word **is not** in the language, the output of the program must be: `"False"`.
- There must not be any other output by the program.

Good luck!

6.2 Solution:

The given grammar has similar problems as in task 5, including left recursion loop and identical prefixes. Here is the improved grammar version:

$$
\begin{aligned}
V_N &= \{S, A, B, F, U, U', X, Y, Z\} \\
V_T &= \{a, d, e, f, n, x, z\} \\
S &= S \\
\Phi &: \\
S &\rightarrow aAXaB \\
A &\rightarrow aA \mid \epsilon \\
X &\rightarrow eX \mid \epsilon \\
B &\rightarrow zUZ \\
U &\rightarrow \epsilon \mid zU' \\
U' &\rightarrow \epsilon \mid a \\
Z &\rightarrow FXYend \\
F &\rightarrow f \mid \epsilon \\
Y &\rightarrow xY \mid \epsilon
\end{aligned}
$$

The implemented code mostly use LL(1) parser, but this is not enough in some cases, such as:
- In $S \rightarrow aAXaB$ part, X can be empty, leading to A to consume all 'a' and leaves no 'a' for later, causing the parser to wrongly reject a valid word like 'aazend'.
- In $Z \rightarrow FXYend$ part, Y can be empty, leading to X to oversume all 'e', same error as above.
In these 2 cases, our parser becomes LL(2) to prevent these errors.